

e Suggestions feature in IDEs contain AI Context only with the open IDE tabs, which adhere to the global AI Context Policy in the root of a repository.

Iteration 2

- In AI Context Retrievers introduce support for Automatic AI Context.
- Connect more features to the AI Context Management system.

Success criteria for the iteration: Prompts sent from the Code Suggestions feature in IDEs contain AI Context with items of Automatic AI Context, which adhere to the global AI Context Policy in the root of a repository.

Iteration 3

- Connect all Duo features on the Web and in IDEs to AI Context Retrievers and adhere to the global AI Context Policy.

Success criteria for the iteration: All Duo features in all environments send AI Context which adheres to the global AI Context Policy

Iteration 4

- Support nested .ai-context-policy.yaml YAML configuration files.

Success criteria for the iteration: AI Context Policy placed into the sub-folders of a repository, override higher-level policies when sending prompts.

Iteration 5

- User-level UI for Supplementary User Context.

Success criteria for the iteration: Users can see and edit the contents of the Supplementary User Context and the context is shared between all Duo features within the environment (Web, IDEs, etc.)

Iteration 6

- Optional: UI for configuring the global AI Context Policy.

Success criteria for the iteration: Users can see and edit the contents of the AI Context Policies in a UI editor.

SECTION: engineering/architecture/design-documents/ai_context_management/decisions/001_policy_on_the_client.md

Summary

To manage AI Context effectively and ensure flexible and scalable solutions, AI Context Policy Management will reside in the same environment, as the AI Context Retriever, and, as a result, as close to the context fetching mechanism as possible. This approach aims to reduce latency and improve user control over the contextual information sent to AI systems.

Context

The original blueprint outlined the necessity of a flexible AI Context Management system to provide accurate and relevant AI responses while addressing security and trust concerns. It suggested that AI Context Policy Management should act as a filtering solution between the context resolver and the context fetcher in the AI Context Retriever. However, the blueprint did not specify the exact location for the AI Context Policy Management within the system.

During a sync discussion, it was determined that placing the AI Context Policy Management close to AI Context Retriever would provide significant benefits. This decision aligns with our approach of having shared components, like the AI Gateway and the Duo Chat UI, to ensure consistency and reduce redundancy across different environments.

Decision

AI Context Management will happen as close to the user's interaction with Duo features as possible. As a result, the AI Gateway will only receive context that is policy-compliant.

Users interact with Duo features in many different environments, including their IDE and the GitLab Web UI. Rather than retrieving the context from this environment and sending it to the AI Gateway for filtering based on the AI Context Policy, this decision states that the AI Context Retriever will filter this content before it reaches the AI Gateway.

This decision allows for better security, flexibility and scalability, enabling dynamic user interactions and immediate feedback on context validation.

Consequences

- Implementation Complexity: Users must create, modify, and remove context policies in each environment where they are interacting with Duo features. This requires multiple implementations to support different environments.
- Flexibility and Scalability: Storing AI Context Policy Management close the AI Context

Retriever allows for more flexible and scalable policy implementations tailored to specific environments, such as IDEs and the Web.

- Reduced Latency: Filtering out unwanted context at the earliest possible stage reduces latency and ensures that only the necessary information is sent to the AI models.
- User Experience: This approach facilitates dynamic UX, providing instant feedback to users in case of failed context validation. Users can manage their supplementary context more effectively through a user-friendly interface.
- Security: By managing policies closer to the content retrieving mechanism, sensitive information can be filtered out locally, enhancing security and user trust.

SECTION: engineering/architecture/design-documents/ai_evaluation_consolidation/_index.md

<!-- Design Documents often contain forward-looking statements -->

<!-- This renders the design document header on the detail page, so don't remove it-->

{{< engineering/design-document-header >}}

Summary

This blueprint proposes consolidating our evaluation tooling ELI5 and

Prompt Library

to create a unified evaluation solution and an accessible way for teams to efficiently evaluate their

AI-powered features. As the main outcome, the blueprint presents the idea of merging ELI5 and Prompt Library

into the Prompt Library repository and ultimately transforming Prompt Library into an engine, retaining the best features of both products.

Motivation

Over time, GitLab developed two evaluation tools that address the same problem of evaluating AI features but

solve this problem differently: Prompt Library, focused on dealing with large scale evaluation, and ELI5,

focused on allowing developers to run quick evaluations.

Overview of Prompt Library

Prompt Library is a feature-rich evaluation tool primarily designed for large-scale evaluations and managing

large datasets. It was built based on Apache Beam and can be run both locally or in a GCP

project using the Dataflow executor. The diagram below demonstrates the internal design of Prompt Library:

Diagram source

Prompt Library provides a CLI interface to run Apache Beam evaluation pipelines that read datasets from BigQuery tables and output results back to BigQuery. Prompt Library supports a rich configuration system through JSON configuration files that allow engineers to tune various parameters related to LLM judges and other heuristics.

Limitations: Prompt Library components are tightly coupled to the Apache Beam API. Any evaluator requires a deep understanding of the Apache Beam API and BigQuery. Please note that Apache Beam is initially designed for scaled processing, and its programming logic may differ from pure Python development. For example, Apache Beam doesn't support sorting in the classical sense. This complexity in extending and updating Prompt Library components is currently causing confusion among teams and creating a barrier to adoption for simpler evaluation tasks.

Overview of ELI5

ELI5 is an evaluation tool built on top of LangSmith, primarily designed for fast prototyping and building evaluation pipelines with less complex component interaction. ELI5 runs locally and requires developers to clone the repository and install dependencies using poetry. If developers need to run ELI5 remotely, they need to perform additional manual steps. The diagram below demonstrates the internal design of ELI5:

Diagram source

Similar to Prompt Library, ELI5 provides a CLI interface to run evaluations, reading data from LangSmith and outputting results back to LangSmith. LangSmith provides a rich UI that allows developers to effectively analyze results and compare experiments to identify metric drift. The ELI5 CLI supports passing arguments to configure evaluation pipelines. Most of the arguments have default values to reduce the burden on developers

and provide earlier results. To further help developers with prototyping evaluation pipelines, ELI5 provides basic building blocks to build evaluators with a unified signature, that is, compare actual output with expected. Depending on the developer's use case, these evaluators can be a pure Python function, LLM judge, or another LangChain runnable.

Limitations: ELI5 solely relies on LangSmith and is limited by its capabilities. ELI5 helps developers efficiently implement evaluators; however, the execution is out of our control, which may be required for advanced use cases.

Conclusion: Moving Towards a Unified Evaluation Approach

Developing both projects simultaneously is inefficient. It increases the need for maintainers, which is already a bottleneck, and creates barriers for contributions. Additionally, it fosters unnecessary competition and repetition between the tools.

Both projects have their own pros and cons. Prompt Library is a powerful tool for custom evaluation at scale but is not flexible for company-wide contributions and adding new changes. On the other side, ELI5 is a user-friendly solution focused on building the evaluation logic with LangSmith UI support to investigate results. At the same time, ELI5 is limited to the LangSmith platform in terms of using it as an engine. Therefore, consolidating our evaluation tooling is important to keep delivering high-quality results.

Goal

Consolidate evaluation tooling to provide a user-friendly and transparent way for teams to evaluate their AI-powered features.

Objectives

Based on the goal and motivation, we define the following objectives:

1. Integrate ELI5 into Prompt Library by converting Prompt Library into an engine: This integration is crucial for creating a unified, powerful, and user-friendly Centralized Evaluation Framework (CEF). The goal is to create a flexible foundation that allows developers to focus on building pure Python function evaluators and running them on the Prompt Library engine, similar to LangSmith.

1. Enhance flexibility, extensibility, and user-friendliness: This objective ensures that the consolidated tool is adaptable to diverse needs and accessible to developers with different skill sets. The result is reducing barriers to entry but offering the power and flexibility to meet the diverse needs of different teams, from quick prototyping to extensive evaluations.

1. Improve documentation and user guidance: Clear documentation and user guidance are essential for successful adoption and effective use of the consolidated tool. This reduces the learning curve, minimizes user errors, and enables teams to leverage the full potential of the evaluation capabilities.

As an outcome, the consolidated tool after completing the defined objectives can be represented as follows:

Diagram source

Non-goals

Dataset management

Dataset management and creation are not considered primary objectives within the scope of this proposal. These aspects are addressed in a separate issue:

<<https://gitlab.com/gitlab-org/modelops/ai-model-validation-and-research/ai-evaluation/prompt-library/-/issues/349>>.

In this proposal, we assume that the necessary data for running evaluations is already available in the dataset Single Source of Truth (SSOT), which is accessible to the consolidated solution.

AI Gateway integration

Centralizing model access through AI Gateway enhances consistency, reduces maintenance overhead, and minimizes the risk of errors associated with managing multiple model access points. Connecting AI Gateway may improve overall system reliability and efficiency. However, integrating AI Gateway into the consolidated tool doesn't appear to be a strict requirement to achieve the main goal. We plan to explore this task further once ELI5 and Prompt Library are successfully integrated.

Apache Beam as Prompt Library executor

Prompt Library relies on Apache Beam to run the evaluation pipelines. Apache Beam was originally designed for

other tasks such as processing at scale and therefore is sometimes a bottleneck when implementing evaluation logic. The Model Validation group has already been looking into replacing Apache Beam with another solution to run the evaluation pipelines. However, to better manage technical debt at this stage and consolidate tooling with fewer code base changes, we have decided to continue using Apache Beam. We are designing the consolidated tool in a way that keeps evaluators simple and decoupled from the Prompt Library's internal logic. This approach will allow us to revisit the rationale behind using Apache Beam once we complete the consolidation work.

Implementation steps

This section describes the implementation steps we need to perform to achieve the objectives defined in the previous section. Please note that we define the implementation steps following GitLab Values - Collaboration, Efficiency, Iteration, and Transparency. Every step can be later converted into an epic or an individual issue depending on complexity.

1. Move ELI5 as a folder into the Prompt Library repository

This step consolidates code without immediate alterations, facilitating the identification of duplicate code and reducing the number of required maintainers. It also increases the visibility of Prompt Library while maintaining existing functionality for current users of both tools.

The process involves relocating the ELI5 folder structure into the Prompt Library repository and updating documentation to reflect the new repository location.

User Impact:

- Existing functionality remains unchanged for both ELI5 and Prompt Library users.
- ELI5 users clone the Prompt Library repository instead of the separate ELI5 repository.
- The ELI5 repository is archived and no longer receives new updates.

Estimated time: < 1 milestone.

2. Convert Prompt Library into an engine

This step focuses on preparing the Prompt Library code base for integration with the existing ELI5 logic. The goal is to convert Prompt Library into a basic engine that accepts ELI5-compatible evaluators.

The process involves updating the Prompt Library code base, providing a Python API similar to LangSmith used

by ELI5. This conversion allows developers to focus on pure evaluation logic without needing to interact directly with Apache Beam. We aim to keep Prompt Library's key strength, that is, customization and running at scale.

User Impact:

- Existing functionality remains unchanged for both ELI5 and Prompt Library users.
- Prompt Library maintainers need to support the previous old highly-coupled evaluation pipelines, fixing bugs only.
- If new evaluation logic needs to be implemented, developers implement ELI5-compatible evaluators and use them with the new Prompt Library engine.

Estimated time: 1-1.5 milestone.

3. Power existing ELI5 evaluators with the Prompt Library engine

This step adds an additional option for ELI5 developers to run their existing evaluators with the Prompt Library engine. By completing this step, we connect the ELI5 codebase to Prompt Library, allowing developers to prototype their evaluation pipelines quickly and also implement advanced custom logic if required.

This process involves updating the ELI5 codebase to include an option of triggering the Prompt Library engine.

User Impact:

- Existing functionality remains unchanged for both ELI5 and Prompt Library users.
- ELI5 users receive an additional option to run evaluators on the custom Prompt Library engine if they require.
- ELI5 users interact with Prompt Library as an engine using the Python API similar to LangSmith.

Estimated time: 1 milestone.

> Note: This step can be implemented in parallel with converting Prompt Library evaluators to be ELI5 compatible.

4. Convert Prompt Library evaluators to be ELI5 compatible

This step converts the existing Prompt Library evaluators to be ELI5 compatible. The goal is to streamline existing old logic, further merging code bases and mitigating any differences between the two

approaches.

This process involves updating the Prompt Library code base. ELI5 already provides basic building blocks to implement various evaluators, including pairwise evaluators or LLM judges. These building blocks can be reused to make corresponding changes to the existing Prompt Library evaluators.

User Impact:

- Existing functionality remains unchanged for both ELI5 and Prompt Library users.
- Prompt Library maintainers support the previous old highly-coupled evaluation pipelines, fixing bugs only.
- If new evaluation logic needs to be implemented, developers implement ELI5-compatible evaluators and use them with the new Prompt Library engine.

Estimated time: 1-1.5 milestone.

- > Note: This step can be implemented in parallel with powering existing ELI5 evaluators with the Prompt Library engine.

5. Merging CLI interface

Both ELI5 and Prompt Library provide CLI interfaces but with different configuration systems created for different purposes. This step ensures that developers run a single CLI interface with a configuration system that provides default values to run evaluations and is also flexible enough for advanced usage when developers need to update evaluation settings.

This process involves updating the CLI interfaces of both ELI5 and Prompt Library and creating a new configuration system based on YAML files with examples and default configurations to support most of the usual developer needs.

User Impact:

- Existing functionality remains unchanged for both ELI5 and Prompt Library users.
- ELI5 and Prompt Library start providing a single CLI interface to run evaluations.
- ELI5 and Prompt Library maintainers support the old CLI interfaces, freezing any additions.

Estimated time: 1 milestone.

6. Deprecate old ELI5 and Prompt Library logic

This step focuses on deprecating and removing the old evaluation pipelines and related

components the maintainers had to support in the previous steps. The goal is to start using the consolidated tool that includes ELI5-compatible evaluators running on the Prompt Library as an engine, the modified CLI, and enhanced configuration system.

User Impact:

- Developers start using ELI5 and Prompt Library as a consolidated tool, that is CEF.
- Developers rely on the new documentation and guidelines for working with the consolidated tool.
- Deprecated dependencies coming from the original ELI5 and Prompt Library projects are successfully cleaned up.

Estimated time: < 1 milestone.

Alternatives

This section presents alternatives that we explored but didn't pursue due to the reasons specified.

1. Build upon ELI5 and archive Prompt Library ELI5 has attracted more contributors and better supports the current needs of our teams. Instead of merging ELI5 into Prompt Library, we could shelve Prompt Library and expand ELI5 with additional features. However, taking the learnings from ELI5, making Prompt Library more user-friendly is an easier task than applying the scale of Prompt Library to ELI5.

1. Archive ELI5 and Prompt Library and start from scratch. This would allow us to take the learnings from ELI5 and Prompt Library without inheriting their technical debt. However, starting a new solution would take longer to implement and might still lead to unforeseen issues. This approach wouldn't necessarily save us from technical debt in the long run.

Future evolution

This section presents future ideas we are considering after achieving the main goal of this proposal.

- Merge Prompt Registry and evaluation repositories. Currently, prompts are being migrated from GitLab-rails into the AI Gateway repo. This is already an improvement over the current setup, but having prompts and evaluations in separate repos is like having GitLab and GitLab unit tests in different repos. Tests

(Evaluations) should be run in the same place as the code (prompts). We suggest further moving them together.

- Evaluation through pipeline CI An alternative user-friendly solution for running evaluations is through

GitLab pipelines, as showcased in this prototype:

<<https://gitlab.com/gitlab-org/ai-powered/custom-models/evaluations/-/jobs/7518078099>>.

Having a

consolidated evaluation repository will simplify the implementation of such a feature.

SECTION: engineering/architecture/design-documents/ai_gateway/_index.md

{{< engineering/design-document-header >}}

Summary

The AI-gateway is a standalone-service that will give access to AI features to all users of GitLab, no matter which instance they are using: self-managed, dedicated or GitLab.com.

Initially, all AI-gateway deployments will be managed by GitLab (the organization), and GitLab.com and all GitLab self-managed instances will use the same gateway. However, in the future we could also deploy regional gateways, or even customer-specific gateways if the need arises.

The AI-Gateway is an API-Gateway that handles traffic steered to it from a globally reachable `cloud.gitlab.com/ai/` route. IDEs currently use `cloud.gitlab.com/ai/` indirectly through a GitLab Rails instance. In

future we plan to allow network endpoints such as IDEs to connect directly to `cloud.gitlab.com` (for more information, see [directly connected clients](#)).

The AI-Gateway then directs this traffic to other services, in this case AI-providers and their models. This north/south traffic pattern allows us to control what requests go where and to translate the content of the redirected request where needed.

Currently, multi-region deployment is not supported; it's a feature under consideration. The existing diagram illustrates a potential architecture for deploying across multiple regions.

Diagram source

By using a hosted service under the control of GitLab we can ensure that we provide all GitLab instances with AI features in a scalable way. It is easier to scale this small stateless service, than scaling GitLab Rails with its dependencies (database, Redis).

It allows users of self-managed installations to have access to features using AI without them having to host their own models or connect to 3rd party providers.

Language: Python

The AI-Gateway was originally started as the "model-gateway" that handled requests from IDEs to provide Code Suggestions. It was written in Python.

Python is an object oriented language that is familiar enough for Rubyists to pick up through in the younger codebase that is the AI-gateway. It also makes it easy for data- and ML-engineers that already have Python experience to contribute.

Directly connected clients

Direct connections are not supported yet and this work is tracked in this epic.

Decision: ADR-001: Allow direct connections

Code completion requests will be sent directly from a client to AI Gateway to improve latency of these requests. It will be up to the client to decide if a request can be sent directly to AI Gateway or to GitLab Rails for additional enrichment. Usage of direct connections will be optional and backward compatible. If a client will not support detection of completion requests, it can still send these requests through GitLab Rails (as is now) without specifying the type of code suggestion request.

API

Basic stable API for the AI-gateway

Because the API of the AI-gateway will be consumed by a wide variety of clients, it is important that we design a stable, yet flexible API.

To do this, we can implement an API-endpoint per use-case we build. This means that the interface between client and the AI-gateway is one that we build and own. This ensures future scalability, composability and security.

The API is not versioned, but is backward compatible. See cross version compatibility for details. The AI-gateway will support the last 2 major versions. For example when working on GitLab 17.2, we would support both GitLab 17 and GitLab 16.

Because clients can connect directly to AI Gateway, common functionality like rate-limiting, circuit-breakers and secret redaction should be added both at this level of the stack as well as in GitLab Rails.

Protocol

The communication between the AI-Gateway service and its clients (including the GitLab Rails application) shall use a JSON-based API.

The AI-Gateway API shall expose single-purpose endpoints responsible for providing access to different AI features. A later section of this document provides detailed guidelines for building specific endpoints.

The AI Gateway communication protocol shall only expect a rudimentary envelope that wraps all feature-specific dynamic information. The proposed architecture of the protocol allows the API endpoints to be version agnostic, and the AI-Gateway APIs compatible with multiple versions of GitLab (or other clients that use the gateway).

This means that all clients regardless of their versions use the same set of AI-Gateway API feature endpoints. The AI-gateway feature endpoints have to support different client versions, instead of creating multiple feature endpoints per different supported client versions.

We can however add a version to the path in case we do want to evolve a certain endpoint. It's not expected that we'll need to do this often, but having a version in the path keeps the option open. The benefit of this is that individual GitLab milestone releases will continue pointing to the endpoint version it was tested against at the time of release, while allowing us to iterate quickly by introducing new endpoint versions.

We also considered gRPC as a protocol for communication between GitLab instances, JSON API, and gRPC differ on these items:

gRPC	REST + JSON
----- -----	
----- -----	
----- -----	
+ Strict protocol definition that is easier to evolve versionless	
- No strict schema, so the implementation needs to take good care of supporting multiple versions	
+ A new Ruby-gRPC server for vscode: likely faster because we can limit dependencies to load (modular monolith)	- Existing Grape API for vscode: meaning slow boot time and unneeded resources loaded
+ Bi-directional streaming	
- Straight forward way to stream requests and responses (could still be added)	
- A new Python-gRPC server: we don't have experience running gRPC-Python servers	
+ Existing Python fastapi server, already running for Code Suggestions to extend	
- Hard to pass on unknown messages from vscode through GitLab to ai-gateway	
+ Easier support for newer VS Code + newer AI-gateway, through old GitLab instance	

- |
- | - Unknown support for gRPC in other clients (vscode, jetbrains, other editors)
- | + Support in all external clients
- |
- | - Possible protocol mismatch (VSCode --REST--> Rails --gRPC--> AI gateway)
- | + Same protocol across the stack
- |

Discussion: Because we chose REST+JSON in this iteration to port features that already partially exist does not mean we need to exclude new features using gRPC or Websockets. For example: Chat features might be better served by streaming requests and responses. Since we are suggesting an endpoint per use-case, different features could also opt for different protocols, as long as we keep cross-version compatibility in mind.

Single purpose endpoints

For features using AI, we prefer building a single purpose endpoint with a stable API over the provider API we expose as a direct proxy.

Some features will have specific endpoints, while others can share endpoints. For example, Code Suggestions or chat could have their own endpoint, while several features that summarize issues or merge requests could use the same endpoint but make the distinction on what information is provided in the payload.

Our goal is to minimize code that we can't update on a customer's behalf, which means avoiding hard-coding AI-related logic in the GitLab monolith codebase:

- We want to minimize the time required for customers to adopt our latest features, and
- We want to retain the flexibility to make changes in our product without breaking support for a long-tail of older instances.
- We want to be able to serve all GitLab distributions (.com SaaS, Self-managed, and Dedicated) with minimal complexity.
- We want to isolate AI capabilities to reduce risk and have a unified control plane.
- We want to provide a unified implementation that can support our "best-in-class" multi-model ensemble approach allowing us to easily support many AI models and AI vendors.
- We want a single point for controlling and measuring cost.
- As much as possible, we want to track metrics (usage statistics, failures to respond, usage pattern, question categories, etc.) in the gateway rather than distributed across many points. (Of course some metrics can only be captured on the client side.)

Having the business logic in GitLab Rails requires customers to upgrade their GitLab instances, which affects the first point. Some of the on-premises users cannot upgrade their instances immediately due to their company policy.

For example, if we had a bug in a prompt template in GitLab Rails and fixed it in 16.6, and customers are using 16.5 and the next upgrade is scheduled in 3 months, they have to use the buggy feature for 3 months.

This does not mean that prompts need to be built inside the AI-gateway. But if prompts are part of the payload to a single purpose endpoint, the payload needs to specify which model they were built for along with other metadata about the prompts. By doing this, we can gracefully degrade or otherwise try to support the request if one of the prompt payloads is no longer supported by the AI gateway. It allows us to potentially avoid breaking features in older GitLab installations as the AI landscape changes.

The AI-Gateway API protocol

It is important to build each single-purpose endpoint, in a version-agnostic way so it can be used by different GitLab instances and by external clients. To achieve this goal:

The AI-Gateway protocol shall rely on a simple JSON envelope wrapping all feature-specific information. The AI-Gateway protocol can be seen as a transport layer protocol from the OSI model (eg: TCP, UDP) which defines how to transport information between nodes, without being aware of what information is being transported.

The AI-Gateway protocol does not specify which information received by single-purpose endpoint should be processed and in which way. Providing endpoint with the freedom to decide if they will use data coming from each protocol envelope or ignore it.

The AI-Gateway protocol defines each request in the following way:

1. Each single-purpose endpoint shall accept requests containing a single JSON object with a single key: `promptcomponents`.
1. The `promptcomponents` key shall contain an array of JSON envelopes that are built according to the following rules:

Each JSON envelope contains 3 elements:

1. `type`: A string identifier specifying a type of information that is being presented in the envelopes
payload. The AI-gateway single-purpose endpoint may ignore any types it does not know about.
1. `payload`: The actual information that can be used by the AI-Gateway single-purpose endpoint to send requests to 3rd party AI services providers. The data inside the payload element can differ depending on the type, and the version of
the client providing the payload. This means that the AI-Gateway
single-purpose endpoint must consider the structure and the type of data present inside the payload optional, and gracefully handle missing or malformed information.
1. `metadata`: This field contains information about a client that built this `promptcomponents` envelope. Information from the metadata field may, or may not be used by GitLab for
telemetry. The same as with the payload all fields inside the metadata shall be considered optional.

The only envelope field that is expected to likely change often is the payload one. There we need to make sure that all fields are optional and avoid renaming, removing, or repurposing fields.

To document and validate the content of payload we can specify their format using JSON-schema.

An example request according to the AI-Gateway component looks as follows:

```
json
{
  "promptcomponents": [
    {
      "type": "prompt",
      "metadata": {
        "source": "GitLab EE",
        "version": "16.7.0-pre",
      },
      "payload": {
        "content": "...",
        "params": {
          "temperature": 0.2,
          "maxOutputTokens": 1024
        },
        "model": "code-gecko",
        "provider": "vertex-ai"
      }
    },
    {
      "type": "editorcontent",
      "metadata": {
        "source": "vscode",
        "version": "1.1.1"
      },
      "payload": {
        "filename": "application.rb",
        "beforecursor": "require 'activerecord/railtie'",
        "aftercursor": "\nrequire 'actioncontroller/railtie'",
        "openfiles": [
          {
            "filename": "app/controllers/applicationcontroller.rb",
            "content": "class ApplicationController < ActionController::Base..."
          }
        ]
      }
    }
  ]
}
```

Another example use case includes 2 versions of a prompt passed in the promptcomponents payload. Where each version is tailored for different 3rd party AI model provider:

```
json
```



```

{
  promptcomponents: [
    {
      "type": "prompt",
      "metadata": {
        "source": "GitLab EE",
        "version": "16.7.0-pre",
      },
      "payload": {
        "content": "You can fetch information about a resource called an issue...",
        "params": {
          "temperature": 0.2,
          "maxOutputTokens": 1024
        },
        "model": "text-bison",
        "provider": "vertex-ai"
      }
    },
    {
      "type": "prompt",
      "metadata": {
        "source": "GitLab EE",
        "version": "16.7.0-pre",
      },
      "payload": {
        "content": "System: You can fetch information about a resource called an
issue...\n\nHuman:",
        "params": {
          "temperature": 0.2,
        },
        "model": "claude-2",
        "provider": "anthropic"
      }
    }
  ]
}

```

Cross-version compatibility

When renaming, removing, or repurposing fields inside payload is needed, a single-purpose endpoint that uses the affected envelope type must build support for the old versions of a field in the gateway, and keep them around for at least 2 major versions of GitLab.

A good practice that might help support backward compatibility: provide building blocks for the prompt inside the promptcomponents, rather than a complete prompt. By moving responsibility of compiling the prompt out of building blocks and into the AI-Gateway, more flexible prompt adjustments are possible in the future.

Example feature: Code Suggestions

For example, a rough Code Suggestions service could look like this:

plaintext

POST /v3/code/completions

json

```
{
  "promptcomponents": [
    {
      "type": "prompt",
      "metadata": {
        "source": "GitLab EE",
        "version": "16.7.0-pre",
      },
      "payload": {
        "content": "...",
        "params": {
          "temperature": 0.2,
          "maxOutputTokens": 1024
        },
        "model": "code-gecko",
        "provider": "vertex-ai"
      }
    },
    {
      "type": "editorcontent",
      "metadata": {
        "source": "vscode",
        "version": "1.1.1"
      },
      "payload": {
        "filename": "application.rb",
        "beforecursor": "require 'activerecord/railtie'",
        "aftercursor": "\nrequire 'actioncontroller/railtie'",
        "openfiles": [
          {
            "filename": "app/controllers/applicationcontroller.rb",
            "content": "class ApplicationController < ActionController::Base..."
          }
        ]
      }
    }
  ]
}
```

A response could look like this:

```
json
{
  "response": "require 'something/else'",
  "metadata": {
    "identifier": "deadbeef",
    "model": "code-gecko",
    "timestamp": 1688118443
  }
}
```

The metadata field contains information that could be used in a telemetry endpoint on the AI-gateway where we could count suggestion-acceptance rates among other things.

The way we will receive telemetry for Code Suggestions is being discussed in 415745.
We will try to come up with an architecture for all AI-related features.

Exposing AI providers

A lot of AI functionality has already been built into GitLab Rails that currently builds prompts and submits this directly to different AI providers. At the time of writing, GitLab has API-clients for the following providers:

- Anthropic
- Vertex

To make these features available to self-managed instances, we should provide endpoints for each of these that GitLab.com, self-managed or dedicated installations can use to give these customers to these features.

In a first iteration we could build endpoints that proxy the request to the AI provider. This should make it easier to migrate to routing these requests through the AI-Gateway. As an example, the endpoint for Anthropic could look like this:

```
plaintext
POST /internal/proxy/anthropic/(endpoint)
```

The endpoint means that the client specifies what is going to be called, for example `/v1/complete`. The request body is entirely forwarded to the AI provider. The AI-gateway makes sure the request is correctly authenticated.

Having the proxy in between GitLab and the AI provider means that we still have control over what goes through to the AI provider and if

the need arises, we can manipulate or reroute the request to a different provider. Doing this means that we could keep supporting the features of older GitLab installations even if the provider's API changes or we decide not to work with a certain provider anymore.

I think there is value in moving features that use API providers directly to a feature-specific purpose built API. Doing this means that we can improve these features as AI providers evolve by changing the AI-gateway that is under our control. Customers using self-managed or dedicated installations could then start getting better AI-supported features without having to upgrade their GitLab instance.

Features that are currently experimental

can use these generic APIs, but we should aim to convert to a single purpose API endpoint before we make the feature generally available for self-managed installations. This makes it easier for us to support features long-term even if the landscape of AI providers change.

API in GitLab instances

This is the API that external clients can consume on their local GitLab instance. For example VSCode that talks to a self-managed instance.

These versions could also widely defer: it could be that the VSCode extension is kept up-to-date by developers. But the GitLab instance they use for work is kept a minor version behind. So the same requirements in terms of stability and flexibility apply for the clients as for the AI gateway.

In a first iteration we could consider keeping the current REST payloads that the VSCode extension and the Web-IDE send, but direct it to the appropriate GitLab installation. GitLab Rails can wrap the payload in an envelope for the AI-gateway without having to interpret it.

When we do this then the GitLab-instance that receives the request from the extension doesn't need to understand it to enrich it and pass it on to the AI-Gateway. GitLab can add information to the promptcomponents and pass everything that was already there straight through to the AI-gateway.

If a request is initiated from another client (for example VSCode), GitLab Rails needs to forward the entire payload in addition to any other enhancements and prompts. This is required so we can potentially support changes from a newer version of the client, traveling through an outdated GitLab installation to a recent AI-gateway.

Discussion: This first iteration is also using a REST+JSON

approach. This is how the VSCode extension is currently communicating with the model gateway. This means that it's a smaller iteration to go from that, to wrapping that existing payload into an envelope. With the added advantage of cross version compatibility. But it does not mean that future iterations also need to use REST+JSON. As each feature would have it's own endpoint, the protocol could also be different.

Authentication & Authorization

GitLab provides the first layer of authorization by authenticating the user and checking if the license permits using the requested feature. This is accomplished using the existing authentication, policy, and license checks built into GitLab.

Authenticating the GitLab instance on the AI-gateway was discussed in:

- Issue 177
- Epic 10808

The specific mechanism by which trust is delegated between end-users, GitLab instances, and the AI-gateway is detailed in the Cloud Connector access control documentation.

The AI Gateway, as a backend service, handles instance authentication among other tasks. It is accessed through the Cloud Connector Load Balancer (currently implemented as Cloudflare), which acts as a Web Application Firewall (WAF) layer but does not perform authentication. The AI Gateway also needs to support end-user authentication because some requests, such as code completion, will be sent directly by clients rather than indirectly through GitLab Rails. A possible solution involving short-term user tokens is described in Epic 13252. The AI Gateway must distinguish between requests proxied by GitLab Rails and direct client requests, as some endpoints or parameters may not be available for direct requests (e.g., clients should only send sub-components of the final prompt, which the AI Gateway will build).

The AI Gateway uses JSON-based API communication and relies on JWTs for authentication. These JWTs are generally scoped to the GitLab instance and are obtained using a PAT or OAuth token. For Code Completions, a short-lived user-bound JWT is used, allowing direct communication between the client and the AI Gateway, bypassing the need to go through the GitLab instance for each request. This short-lived JWT is valid for one hour, reducing the number of JWTs needed for multiple prompt requests.

When a client initiates a prompt in GitLab Duo Chat or an IDE, the prompts, telemetry, context, and JWT token are packaged and transmitted using TLS. It is important to note that the payload itself is not encrypted; it is passed as plain text in JSON format within the request body. This approach is consistent for calls to third-party models where only tunnel-level TLS encryption is used via HTTPS.

The AI Gateway operates within GitLab hosted infrastructure, interfacing with APIs hosted in GitLab accounts for providers like Anthropic and Vertex. Models are hosted as frozen versions of Google's models within GitLab GCP tenancy, using private endpoints within the security perimeter. While prompts from different customers use the same shared model, each session maintains a user-level connection to ensure isolation.

The AI Gateway is designed as a stateless service, meaning it does not store any customer-specific data. Decryption of payloads occurs at the network layer rather than the application layer, and encryption keys are generated per request using GKE native GCP processes. Requests to Anthropic APIs are made via the public internet, whereas requests to Vertex AI models are optimized by being co-located within the same GCP region. All connections are secured with TLS/HTTPS to ensure encrypted communication throughout the data flow.

Embeddings

NOTE:

For the embedding database, see RAG for GitLab Duo.

Embeddings can be requested for all features in a single endpoint, for example through a request like this:

```
plaintext
POST /internal/embeddings
```

```
json
{
  "content": "The lazy fox and the jumping dog",
  "contenttype": "issuetitle",
  "metadata": {
    "source": "GitLab EE",
    "version": "16.3"
  }
}
```

The contenttype and properties content could in the future be used to create embeddings from different models based on what is appropriate.

The response will include the embedding vector besides the used provider and model. For example:

```
json
{
  "response": [0.2, -1, ...],
  "metadata": {
    "identifier": "8badf00d",
    "model": "text-embedding-ada-002",
    "provider": "openai",
  }
}
```

When storing the embedding, we should make sure we include the model and provider data. When embeddings are used to generate a prompt, we

could include that metadata in the payload so we can judge the quality of the embedding.

Deployment

Currently, the model-gateway that will become the AI-gateway is being deployed using from the project repository in `gitlab-org/modelops/applied-ml/code-suggestions/ai-assist`.

It is deployed to a Kubernetes cluster in it's own project. There is a staging environment that is currently used directly by engineers for testing.

In the future, this will be deployed using Runway. At that time, there will be a production and staging deployment. The staging deployment can be used for automated QA-runs that will have the potential to stop a deployment from reaching production.

Further testing strategy is being discussed in [#10563](#).

Self-hosted deployment of AI Gateway

Customers within self-hosted environments can also deploy a self-hosted version of AIGW, enabling usage of Duo features even within airgapped solutions, including those requiring FedRAMP compliance. This approach allows us to serve sophisticated customers capable of managing their own LLM infrastructure, providing them with more control and flexibility. Instructions on how to setup AIGW on self-managed are found in our docs.

While this adds another component to be installed, the alternative would be to implement a model interface in the monolith. This presents several challenges, including:

- Need to reimplement in Ruby all libraries that exist in Python for LLM handling
- Duplicate codebases and maintainance
- Lack of feature parity between self-hosted and .com
- Difficult logging and tracing for self-hosted customers

Relying on AI Gateway for both self-hosted and .com environments on the other hand centralizes the infrastructure challenges and solution when dealing with interfacing with LLMs.

AI Gateway and GitLab version matching

Customers need a clear indication of which AIGW version is compatible to their GitLab instance. Since AI Gateway deployment is not managed by Omnibus/GNG, we achieve this by providing image tags

for each GitLab version: an AIGW image with TAG gitlab-17.2-ee is compatible with GitLab version 17.2-ee.

More discussion on versioning

Alternative solutions

Alternative solutions were discussed in [applied-ml/code-suggestions/ai-assist161](#).

Decisions

- ADR-001: Allow direct connections
- ADR-002: Exposing proxy endpoints to AI providers

Future work

AI Gateway aim is to become the primary method for the monolith to access machine learning models across all usages of GitLab and create a consistent user journey when developing AI-backed features. To do so, these goal is split down into three categories:

- Centralized Access Through AI Gateway
- Unit Primitives

Centralized Access Through AI Gateway

The AI Gateway, a standalone service, is the sole access point for all communication between GitLab installations and third-party AI models. It is designed to centralize and manage access to all GitLab features, whether they are in-app functionalities or code suggestions, irrespective of their deployment methods.

This strategy significantly simplifies enterprise management and abstracts machine learning away from the monolith. With future expansions including telemetry, embeddings API, and multi-region/customer-specific deployments, our goal is to provide a scalable, comprehensive AI solution for all GitLab users, regardless of their installation type.

Model registry is a feature that allows users to use GitLab to manage the machine learning models. While not solely focused on large language models, and currently more targeted at smaller model applications, which could be deployed in various ways: as a standalone library, a service, a pod, a cloud deployment, and so forth. For these user-deployed models, the ability to auto-configure an API that's accessible through the AI Gateway could be a significant feature.

- AI Gateway as the Sole Access Point for Monolith to Access Models

Unit Primitives

Unit Primitives are a fundamental part of our strategy for managing access to AI features through the AI Gateway. They represent the smallest unit of functionality that can be accessed and managed through the Gateway. This approach provides a more granular control over the

functionalities exposed through the AI Gateway and simplifies the management of AI features. It also paves the way for future work on supporting user-deployed models and locally hosted models. From a business perspective, unit primitives are the smallest pieces that may be shuffled across various tiers or packaging models, providing flexibility and adaptability in our offerings.

In the initial iteration, we will support two primitives: Code Suggestions and Chat. The latter will encompass all Chat features in one primitive.

In the next iteration, we plan to decompose the Chat primitive into multiple primitives based on top-level tools. This work is dependent on the completion of the task to move classification into the AI Gateway.

The introduction of Unit Primitives will simplify the management of AI features and provide a more granular control over the functionalities exposed through the AI Gateway. This will also pave the way for future work on supporting user-deployed models and locally hosted models.

For more details on how to implement and integrate unit primitives, refer to the Cloud Connector documentation.

Other components in the AI stack

While AI Gateway centralizes access to AI features and models, it interacts with other components to help users achieve their goals:

- Model registry: manage and deployment machine learning models

Model registry

Model registry is a feature that allows users to use GitLab to manage the machine learning models. While not solely focused on large language models, and currently more targeted at smaller model applications, which could be deployed in various ways: as a standalone library, a service, a pod, a cloud deployment, and so forth. For these user-deployed models, the ability to auto-configure an API that's accessible through the AI Gateway could be a significant feature.

SECTION: engineering/architecture/design-
documents/ai_gateway/decisions/001_direct_connections.md

{{% alert %}}

This decision is scoped to code completion requests. Code completion requests are a type of code suggestion request.

{{% /alert %}}

Summary

Code completion requests will be sent directly from client to AI Gateway, while code generation requests will still be sent indirectly through GitLab Rails.

Context

The original iteration of the blueprint suggested to route all code suggestion requests to AI Gateway indirectly through GitLab Rails.

There were multiple reasons for this:

- the decision if a code suggestion request is a completion or generation request was done on GitLab Rails side
- Simpler authentication of requests could be used. For example, we could use IJWTs to authenticate requests from self-managed instances.

In epic 12224, we discussed various options to decrease request latency which would decrease also overall response time.

Decision

In the epic we decided to use different request flows for code completion and code generation requests. The reason is that both request types are slightly different:

- Code completion requests are much more sensitive to latency and from the request timing breakdown, more than 50% of time is spent connecting to the AI provider.
- Code generation requests are not so sensitive to latency. Most of the request time is spent on AI provider side and latency optimization would have marginal effect in the overall response time. Also we enrich these requests with additional data in GitLab Rails so it's still important to route these requests through GitLab Rails.

For code completion requests we decided that these requests can be sent by clients directly to AI Gateway. This will decrease latency by 150ms. We will benefit from this also when AI Gateway supports multi-region deployments because then code completion requests can be handled inside the same region. GitLab Rails does not support multi-region deployment, so with having completion requests routed through GitLab Rails we cannot eliminate long-distance connection hops which increases latency as well.

Usage of direct connection will be optional optimization. If a client does not recognize that a request is of type "completion", then it can still send the request through GitLab Rails as is now.

We considered also using the websocket protocol, but for now we will keep using HTTP. The plan is to use short-term JWTs to authenticate direct connections. This should be a fast operation (we use this already for requests sent indirectly) and it duplicates the major advantage of websocket. With websocket, we can authenticate users only once when opening the connection. So although with websocket, the connection might be faster by a few milliseconds, it does not outweigh downsides. This means mainly much more work both on AI Gateway and client side to switch to a different protocol. Also websocket is a stateful protocol which would complicate AI Gateway multi-region support.

Latency will improve even further for non-US GitLab users when multi-region deployment of AI-gateway is supported. We then avoid long-distance connection hop client<->GitLab Rails.

Consequences

- No pre-processing for code completions requests by GitLab Rails. We currently do not do any special pre-processing of code completion requests on GitLab Rails side. If we want to do it in future, a possible approach would be to pre-fetch GitLab-specific data and do this enrichment on the client side as outlined in this comment.
- Cloud Connector needs to be able to authenticate these direct connections. We will use short-term JWTs issued for end users. As decided in issue 168 that AI Gateway will issue these short-term tokens.
- To fully leverage this optimization, we will need to support request type recognition (completion vs generation) on the client side. This is already supported for some clients and languages, but ideally we should support it for most of frequently used editors and programming languages.
- Client initialization process needs to be updated. Clients will receive short-term token with other direct connection details from GitLab Rails instance. For more information, see issue 66
- AI Gateway should support monitoring and rate-limiting which is now handled by GitLab Rails.

SECTION: engineering/architecture/design-documents/ai_gateway/decisions/002_proxy.md

Summary

AI Gateway exposes proxy endpoints to AI providers to let existing client libraries in GitLab-Rails access them.

This is a drop-in replacement that should be used until stage groups move to a single purpose endpoint.

We are veering from our ultimate desired architecture in order to bring these features to market for self-managed GitLab instances faster.

Context

The original iteration of the blueprint suggested to have a single purpose endpoint for each AI-powered feature.

There were multiple reasons for this:

- Avoid hard-coding AI-related logic in the GitLab monolith codebase to minimize the time required for customers to adopt our latest features.
- Retain the flexibility to make changes in our product without breaking support for a long-tail of older instances.

In issue 454543, we discussed various options to enable existing AI features in self-managed GitLab.

Decision

In the issue we decided to introduce proxy endpoints to AI providers so that our Ruby client libraries Anthropic::Client and VertexAI::Client work as-is. The reason is that:

- It's challenging to re-write the existing business logic in Python AI Gateway:

- Some of the business logic is using dependencies that are only available in GitLab-monolith (e.g. Feature Flag, Caching in Redis).
This requires us to workaround these implementations, which is error prone.
- Due to the intensive inheritance in Gitlab::LLm namespace, it's hard to extract the actual business logic that are taking an effect.
- We lack a tool to evaluate whether the quality and functionality of the feature remain consistent before and after changes.
- Duo Chat bacame GA regardless of the existing POST /v1/chat/agent endpoint which serves as a proxy endpoint. Technically, this is not a single purpose endpoint yet.

Technical details

Here is the overview of the request flow:

mermaid

flowchart LR

subgraph AIGateway

Proxy["Proxy"]

end

subgraph Provider1["Anthropic"]

direction LR

Model1(["Claude 2.1"])

end

subgraph Provider2["VertexAI"]

direction LR

Model2(["text-bison"])

end

subgraph SM or SaaS GitLab

DuoFeatureA["Duo feature A"]

DuoFeatureB["Duo feature B"]

end

DuoFeatureA -- POST /v1/proxy/anthropic/v1/complete --- Proxy

DuoFeatureB -- POST /v1/proxy/vertex-ai/v1/text-bison:predict --- Proxy

Proxy -- POST /v1/complete --- Provider1

Proxy -- POST /v1/text-bison:predict --- Provider2

Anthropic

Expose the following HTTP/1.1 endpoint in AI Gateway:

plaintext

POST /v1/proxy/anthropic/(path)

path can be forwarded to the following endpoints:

- /v1/complete
- /v1/messages (Future iteration)

Vertex AI

Expose the following HTTP/1.1 endpoint in AI Gateway:

plaintext
POST /v1/proxy/vertex-ai/(path)

path can be forwarded to the following endpoints:

- /v1/{endpoint}:predict
 - endpoint must be one of: chat-bison, code-bison, codechat-bison, text-bison, textembedding-gecko@003.

Common behavior

- Request body is sent to AI providers as-is.
- Request headers are filtered/replaced by AI Gateway accordingly e.g. Allow only accept, content-type, anthropic-version and filter out the rest. x-api-key is added.
- Response body is returned to clients as-is.
- Response headers are filtered/replaced by AI Gateway accordingly e.g. Allow only date, content-type, transfer-encoding and filter out the rest.
- Response status is returned to clients as-is.
- HTTP Streaming is supported.
- if unsupported path is specified, AI Gateway responds with a 404 Not Found error.

Access control

- Clients must send JWT issued by GitLab.com or Customer Dot.
 - This JWT contains scopes that indicates the permissions given to the GitLab-instance. This scopes will vary per Duo subscription tier.
 - To access these proxy endpoints, scopes must include one of: explainvulnerability, resolvevulnerability, generatedescription, summarizealopennotes, generatecommitmessage, summarize review, analyzecijobfailure.
 - Requests that do not meet the specified criteria will result in a 401 Unauthorized Access error.
- Clients must send X-Gitlab-Feature-Usage headers in HTTP requests.
 - This X-Gitlab-Feature-Usage header indicates the purpose of the API request.
 - To access these proxy endpoints, X-Gitlab-Feature-Usage must be one of: explainvulnerability, resolvevulnerability, generatedescription, summarizealopennotes, generatecommitmessage, summarize review, analyzecijobfailure.
 - Requests that do not meet the specified criteria will result in a 401 Unauthorized Access error.
- For logging, we add the value of X-Gitlab-Feature-Usage header in access logs in AI Gateway.
- For metrics, we instrument the concurrent requests with ModelRequestInstrumentator and input/output tokens with TextGenModelInstrumentator in AI Gateway. It should be labeled with X-Gitlab-Instance-Id, X-Gitlab-Global-User-Id and X-Gitlab-Feature-Usage.

- For telemetry, we add Internal Event Tracking for each feature in GitLab-Rails.
Alternatively, we could use the existing snowplow tracker in AI Gateway, which requires additional work for introducing an unified schema.

For further access control improvement, see this issue.

Consequences

- Experimental AI features are enabled on self-managed instances.
- Stage groups can start working on improving the business logic of the feature. This proxy work can be worked in parallel.
- Stage groups don't need to rush refactoring business logic in Python AI Gateway for GA release.
They can take time post-GA.
- We can detect abusers by checking X-Gitlab-Instance-Id, X-Gitlab-Global-User-Id and X-Gitlab-Feature-Usage in logs and metrics.
- We can block abusers by gating the access at Cloud Connector LB (Cloud Flare) or AI Gateway middleware.

SECTION: engineering/architecture/design-documents/ai_model_selection/_index.md

<!-- Design Documents often contain forward-looking statements -->

<!-- This renders the design document header on the detail page, so don't remove it-->
{{< engineering/design-document-header >}}

Background

A growing number of AI models both open-source and proprietary are continually emerging, each with distinct characteristics such as latency, maximum token length, training approach, and output quality. Today, GitLab Duo relies on a fixed set of models for features like Code Suggestions, Chat, Duo Code Review, and Vulnerability Analysis. This rigidity limits administrators, operations teams, and end users in selecting the model that best suits their needs.

Additionally, whenever we introduce a new sub-processor or model, customers must undertake a thorough review and threat assessment process. These procedures that can span several months. This review is crucial for compliance and governance but also slows our ability to adopt new and potentially better-performing models.

In light of these challenges, we need an end-to-end solution that spans multiple feature groups to allow flexible model selection. This blueprint outlines the technical architecture required to integrate model selection options throughout GitLab Duo, providing administrators, operators, and end users with fine-grained control over which models can be used while also streamlining the governance and compliance workflow. For further background, please see Issue 513430.

Current State

Model Switching with Feature Flags:

On gitlab.com and Self-Managed instances, model selection is currently controlled by feature flags. These flags are typically toggled on or off by an administrator or operations user. Because these flags apply at the instance level rather than at a user or group level, individual users and enterprises do not have the freedom to switch between different models based on their specific tasks.

Self-Hosted Model Configuration:

For Self-Managed installations, administrators can configure self-hosted models at an instance level. While this allows some flexibility in choosing or updating models, it still does not permit end users (developers) to select models on a per-feature basis in the UI or IDE. Furthermore, .com customers currently have no governance mechanism to specify which models their organization members can use.

Phases

We can deliver this work in iterations so that we deliver value to the customer incrementally

Iteration 1: Top-level Namespace Configuration: In this phase, customers will be able to select a model for each feature at the root namespace level. This will allow .com customers to decide which models they want their namespace to use.

The model used for a given action will be the one set by the top-level namespace, depending on the context the user is in (project or namespace).

Related epic.

Iteration 2: Consolidating feature release

Related epic.

Future Iterations and consideration

Sub-level Namespace Cascading Configuration:

Child namespaces will be able to assign a specific model for each feature as they did in the top-level iteration. In addition to that, they will be able to select a subset of available models for their downstream namespaces to choose from.

The model used for a given action will be the one set by the closest upstream namespace, including the current namespace, with a feature setting configured, depending on the context the user is in.

We will wait for further demand before considering planning this iteration. Related Issue.

Organization-Level Configuration: In this phase we enable managed model configuration for .com, self-managed and dedicated at the organiz level. Supported models are stored in the AI Gateway. These models will then be retrieved by gitlab.com, Self-managed instances and dedicated instances. This is currently not planned as Organizations are not GA

Future iterations will cover the ability to let users decide which model to use for a specific feature in both the IDE and the GitLab UI. Users wou